

Documentación del proceso de corrección de máscaras para segmentación automática de vértebras T1–L5

1. Objetivo del documento

El objetivo principal fue mejorar la calidad de las máscaras asociadas a las vértebras desde **T1 hasta L5**, manteniendo una convención de **17 clases anatómicas vertebrales**, además del fondo. La corrección se realizó como parte de una fase de curación de datos previa al uso de estas máscaras en modelos de aprendizaje automático.

El documento está pensado como una guía interna para el equipo técnico, docente o profesional que necesite revisar el procedimiento, reproducirlo o verificar las decisiones tomadas durante la corrección.

2. Descripción general del dataset

El dataset contiene radiografías de columna y máscaras asociadas. Las imágenes se identifican con dos prefijos principales:

- **N**: imágenes de columnas normales.
- **S**: imágenes de columnas con escoliosis.

Las máscaras originales se encuentran en la carpeta:

LabelMultiClass_ID_PNG

Los archivos siguen una estructura similar a:

LabelMulti_N_25.png

LabelMulti_S_156.png

Cada máscara es una imagen en la que el fondo corresponde al valor **0**, y las vértebras están codificadas con valores enteros del **1** al **17**.

La convención utilizada es la siguiente:

0 = fondo

```
1 = T1
2 = T2
3 = T3
4 = T4
5 = T5
6 = T6
7 = T7
8 = T8
9 = T9
10 = T10
11 = T11
12 = T12
13 = L1
14 = L2
15 = L3
16 = L4
17 = L5
```

Aunque algunas radiografías pueden mostrar otras estructuras, como vértebras cervicales o estructuras adicionales, el alcance de este proceso se limitó a la identificación y corrección de las vértebras **T1–L5**.

3. Problema identificado

Durante la revisión inicial del archivo:

reporte_por_mascara_version_final.csv

Se observó que varias máscaras no contenían las 17 etiquetas esperadas, aunque en algunos ejemplos se nota visualmente que las imágenes parecían contener las 17 vértebras desde T1 hasta L5. Tomando el ejemplo **LabelMulti_S_156.jpg** de la carpeta

LabelMultiClass_Color_JPG se puede contar que existen 17 vértebras distintas, sin embargo solamente hay 4 “colores” diferentes. En este ejemplo se muestra la imagen original, y justamente después, se agrega la corrección. También se quiere aclarar que para hacer la corrección, NO se utiliza esa imagen, sino la máscara original, algo que se explorará más a detalle en el [Flujo Práctico](#) .

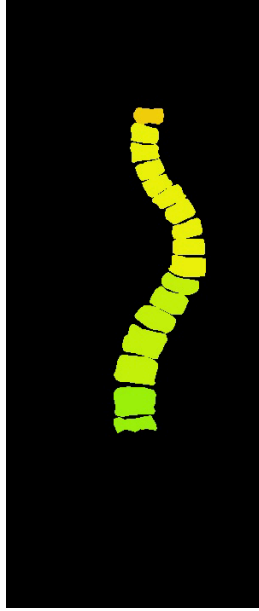


Figura 1. Imagen S_156 Original

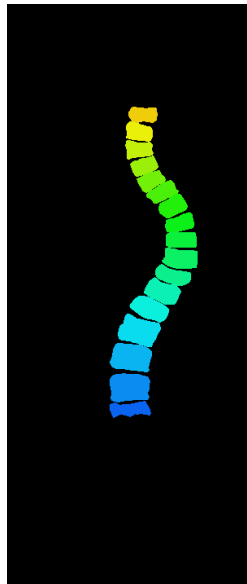


Figura 2. Imagen S156 corrección

El problema principal era que algunas vértebras aparecían visualmente presentes, pero en la máscara estaban:

- fusionadas con otra vértebra bajo el mismo identificador,
- mal diferenciadas por color o ID,
- o no separadas correctamente en la máscara multicategoría.

En otras palabras, la imagen podría mostrar 17 cuerpos vertebrales, pero la máscara podía tener solo 4, 10, 14, 15 o 16 etiquetas correctamente diferenciadas.

Este problema era importante porque las máscaras son la referencia directa para entrenar o evaluar modelos de segmentación. Si una máscara no distingue correctamente las vértebras, el modelo aprende una representación incorrecta o incompleta.

4. Archivos y carpetas principales

Para este proceso se utilizó una carpeta de trabajo llamada:

vertebra_mask_correction

Esta carpeta contiene el script, reportes, salidas editables, máscaras corregidas y logs del proceso.

La estructura relevante es:

```
vertebra_mask_correction/
├── mask_correction_workflow.py
├── reports/
│   └── suspicious_masks_report.csv
├── outputs/
│   ├── LabelMultiClass_Color_EDIT/
│   │   └── LabelMulti_S_156_COLOR_EDIT.png
│   ├── LabelMultiClass_ID_PNG_corrected/
│   │   └── LabelMulti_S_156.png
│   └── logs/
│       └── correction_log.csv
```

La carpeta **LabelMultiClass_Color_EDIT** contiene las versiones en color utilizadas para edición manual. Estas se generan desde la máscara original, y al hacer las modificaciones se reemplaza en la misma ubicación sin ninguna compresión.

La carpeta **LabelMultiClass_ID_PNG_corrected** contiene las máscaras finales corregidas en formato ID.

El archivo **correction_log.csv** registra las conversiones realizadas y permite verificar los cambios entre la máscara original y la corregida.

5. Conversión de máscaras ID a máscaras en color

Las máscaras originales en formato ID son difíciles de revisar visualmente porque los valores enteros de las clases pueden verse casi negros o con diferencias mínimas entre sí.

Por esta razón, se creó una conversión de cada ID a un color RGB fijo. Esta conversión permite visualizar claramente cada vértebra como una región de color diferente.

El script contiene una paleta fija para las 17 vértebras y el fondo. Esta paleta permite convertir una máscara ID en una imagen RGB editable.

Ejemplo general:

```
ID 1 → T1 → amarillo (#F2D10C)
ID 2 → T2 → amarillo/verde (#EBF20C)
...
ID 17 → L5 → azul (#0C68F2)
```

Estas imágenes de color no son las máscaras finales de entrenamiento. Se usan únicamente como herramienta visual y editable para realizar correcciones manuales.

6. Edición manual en GIMP

La edición manual se realizó usando **GIMP**. El flujo visual consistió en abrir la máscara RGB editable y corregir las regiones vertebrales con los colores correspondientes.

En la interfaz de GIMP se mantuvo visible una paleta personalizada con los colores de **T1 a L5**, de manera que cada modificación pudiera hacerse con los colores exactos definidos por el script.

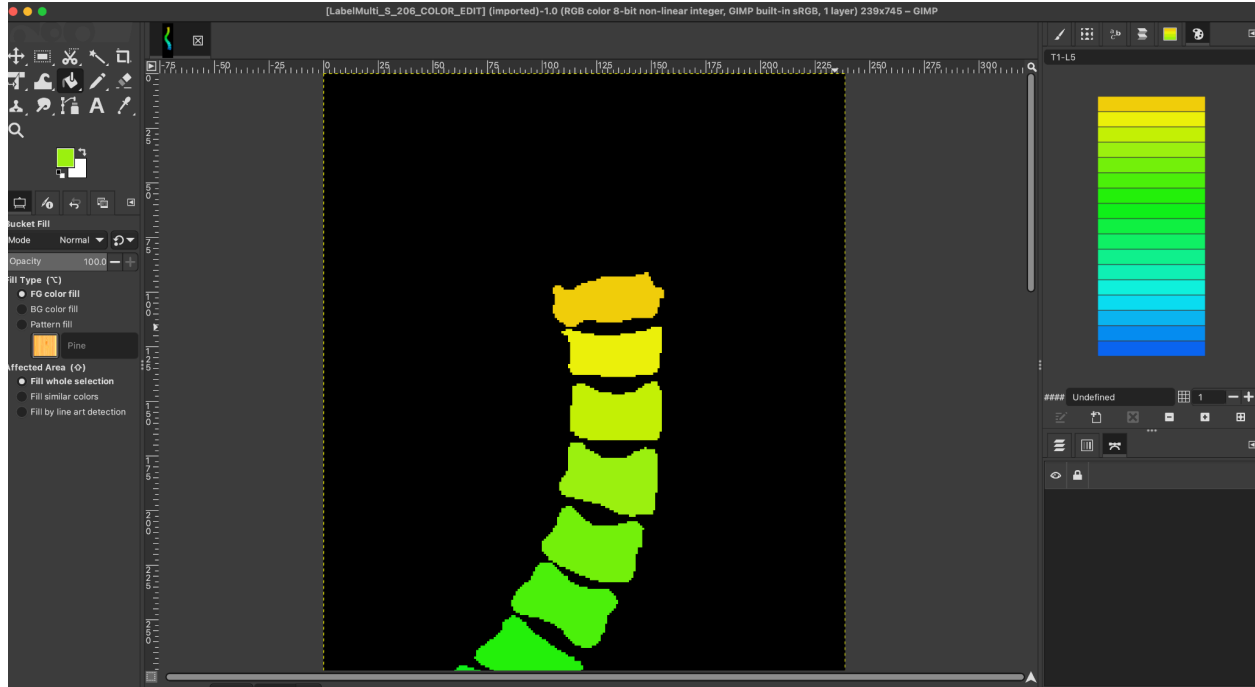


Figura 3. Ejemplo de edición manual utilizando GIMP

En el panel central se observa la máscara RGB editable, mientras que en el panel derecho se mantiene una paleta fija con los colores correspondientes a las clases T1–L5. Esta paleta permite corregir manualmente las vértebras manteniendo consistencia con los IDs definidos en el script.

Durante la edición se evitó usar colores fuera de la paleta. Esto era fundamental porque el script de conversión solo acepta los colores RGB definidos previamente. Si se utiliza un color con antialiasing, opacidad, bordes suaves o una variación mínima del color esperado, el script rechaza la conversión. Se aclara que se cambia manualmente el color de las vértebras según su posición.

7. Validación estricta de colores

Después de editar una máscara en GIMP, la imagen RGB corregida se convertía nuevamente a una máscara ID.

El script realiza una validación estricta:

- revisa todos los colores presentes en la imagen editada,
- verifica que cada color existe en la paleta definida,
- rechaza colores inesperados,
- valida que la máscara final solo contenga IDs entre **0** y **17**,
- y guarda la máscara corregida como imagen de un solo canal.

Esto permitió evitar errores silenciosos. Por ejemplo, si una herramienta de edición generaba un borde con un color ligeramente diferente, el script no permitía guardar la máscara corregida hasta que el problema fuera solucionado. Este error normalmente pasa cuando se utiliza una herramienta del software que tiene una opacidad menor a 1.

8. Flujo práctico de corrección

El flujo general fue el siguiente:

Paso 1: Identificar máscaras sospechosas

Primero se revisó el reporte por máscara para detectar imágenes donde el número de etiquetas era menor al esperado.

Por ejemplo, si una máscara tenía solamente 4 etiquetas (S_156), pero visualmente la imagen mostraba 17 vértebras, esa máscara se marcaba como candidata para corrección. Para identificar si una imagen tiene las 17 vértebras, se cuenta manualmente la máscara de colores para garantizar que están todas las vértebras, en la mayoría de casos se evidencian las 17 vértebras.

Paso 2: Exportar la máscara editable en color

Se utiliza la máscara original para generar la máscara de colores y desde el script se exportaba una versión RGB editable:

```
export_editable_color_mask("LabelMulti_S_156")
```

Esto generaba un archivo como:

```
outputs/LabelMultiClass_Color_EDIT/LabelMulti_S_156_COLOR_EDIT.png
```

Paso 3: Editar manualmente en GIMP

La máscara en color se abría en GIMP.

Luego se corrigen las regiones vertebrales usando la paleta fija T1–L5.

En muchos casos, el error consistía en que varias vértebras consecutivas compartían el mismo color. En esos casos, se asigna manualmente el color correcto a cada vértebra.

Un caso en donde se evidencia claramente que existe un error mínimo es la imagen S_146 en la imagen original (izquierda). En el documento esta imagen solo tenía 16 etiquetas, sin embargo se pueden contar que hay 17 vértebras, al inspeccionar un poquito más se evidencia que entre la vértebra T5 y T6 hay un error, al acercarnos vemos que por solo 1 pixel que

comparten estas vértebras se está identificando como si fuera una sola, es decir que tiene el mismo color.

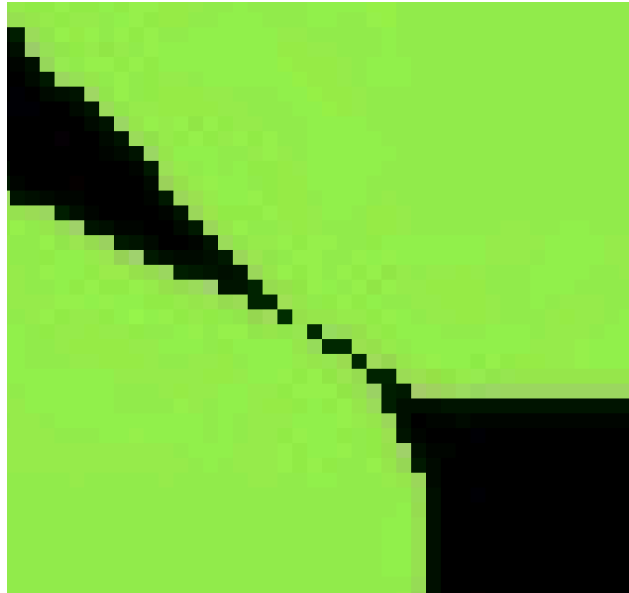
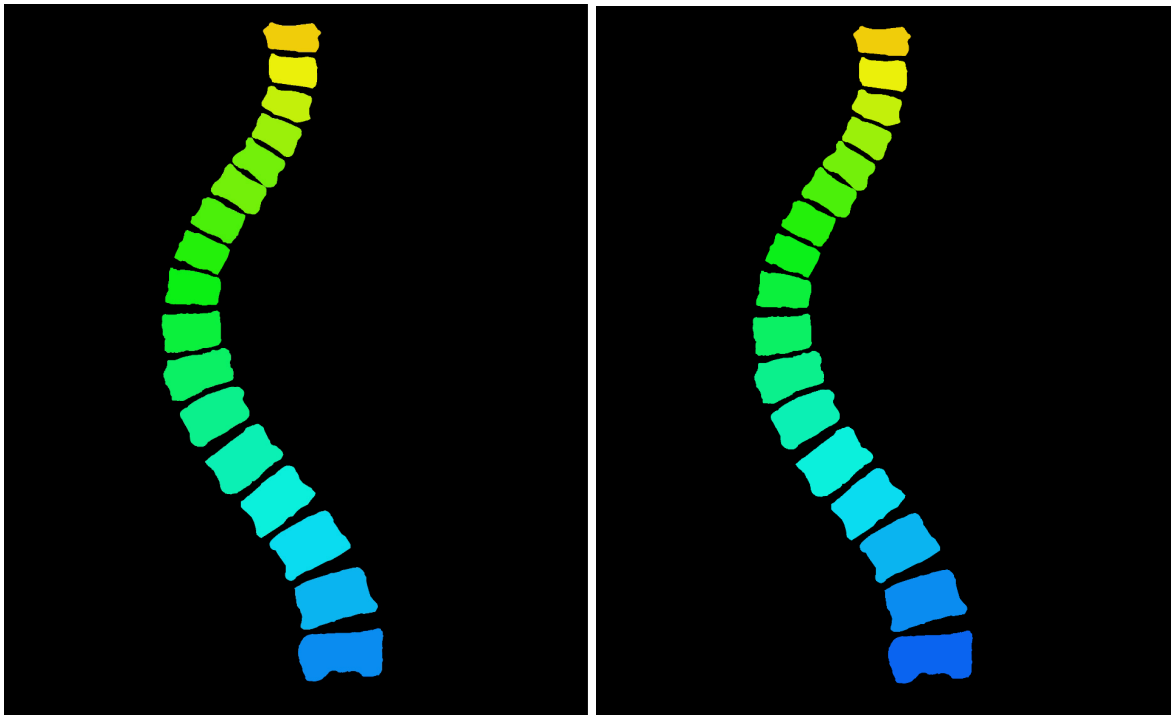


Figura 4. Intercession vertebra T5 T6 imagen S_146

Al hacer esta corrección nos cambia los colores desde T6 hasta L5 y se puede evidenciar que en la imagen corregida (derecha) ahora hay un color de mas que es el L5 (la última vértebra que se está segmentando), y por lo tanto las otras también cambian un poco.



Izquierda: Original - Derecha: Corrección

Paso 4: Convertir la máscara corregida a formato ID

Después de guardar la imagen editada, se ejecutaba:

```
convert_edited_color_to_id("LabelMulti_S_156")
```

Esto generaba la máscara corregida final en:

```
outputs/LabelMultiClass_ID_PNG_corrected/LabelMulti_S_156.png
```

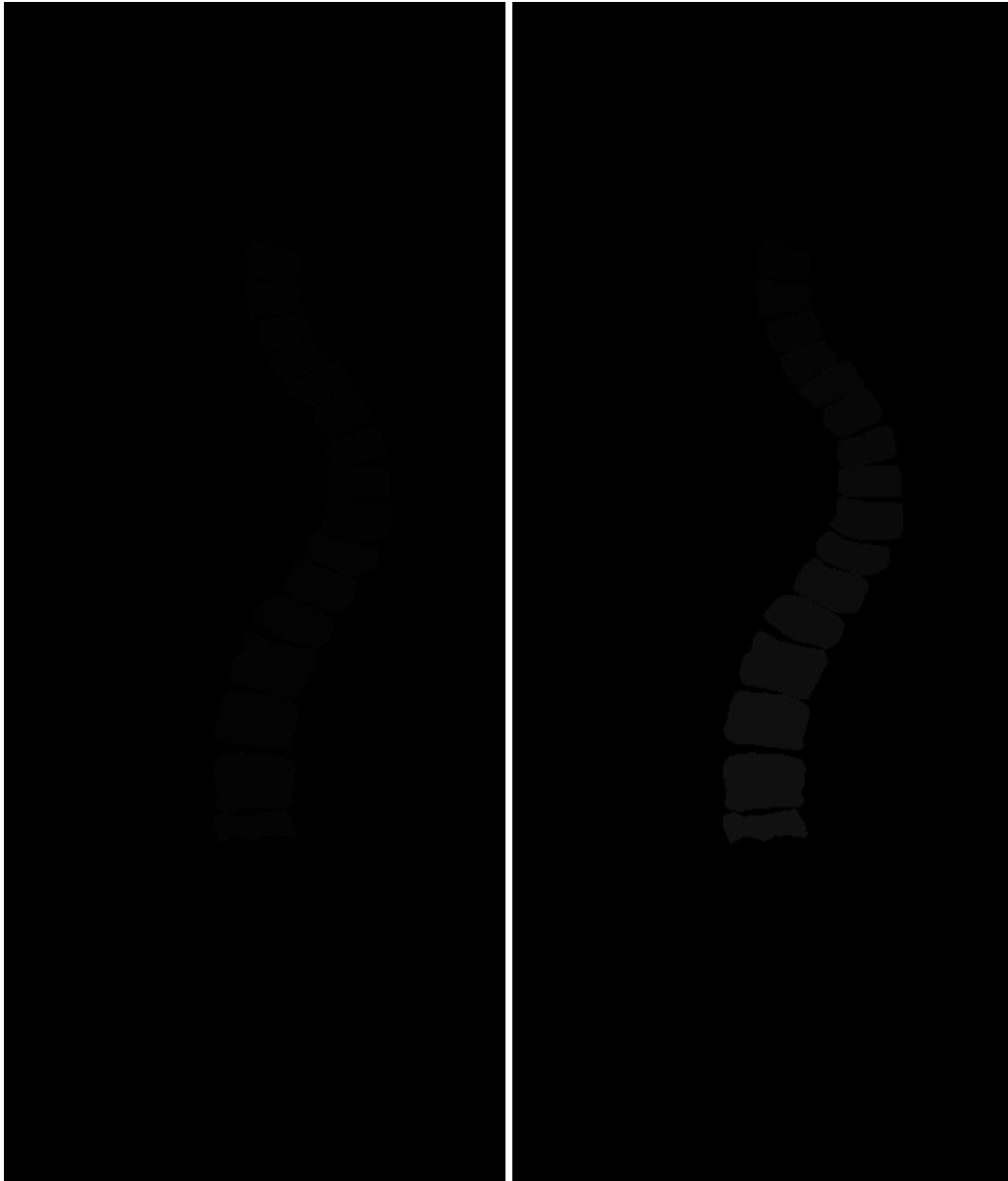
Paso 5: Verificar resultado en consola

El script imprimía los IDs originales y los IDs corregidos.

```
Ejemplo conceptual:  
Original unique IDs: [0, 1, 2, 3, 4]  
Corrected unique IDs: [0, 1, 2, 3, 4, ..., 17]  
Correction applied:  True
```

Esta comparación permitió confirmar que la corrección había sido aplicada correctamente.

Adicionalmente para cada imagen se abre en un navegador con la máscara original y la nueva que se generó, esto debería mostrar una diferencia clara dado que ya existen más etiquetas. En el ejemplo de S_156 se nota un poco mejor ya que pasamos de 4 etiquetas a 17. Este proceso es un poco más complicado dado que las imágenes son difíciles de ver, sin embargo se adjuntan está 2 imágenes, la primera (izquierda) es la máscara original y la segunda (derecha) es la corrección, donde se puede apreciar que ahora se nota la figura un poquito más.



Izquierda: Original - Derecha: Corrección

Paso 6: Registrar la corrección

El resultado también quedaba registrado en:

```
outputs/logs/correction_log.csv
```

Nota: En algunos casos puede aparecer inicialmente **correction_applied = False** debido a un intento previo, error de guardado o conversión incompleta. Sin embargo, cuando se revisa el registro posterior correspondiente, la corrección aparece como aplicada correctamente.

9. Criterio usado durante la corrección

Durante la corrección se asumió que **T1 estaba correctamente identificada** en los casos revisados. Esto sirvió como punto de referencia para asignar los colores de las vértebras siguientes hasta L5.

La corrección siguió estos criterios:

- mantener la convención T1–L5;
- conservar máximo 17 vértebras etiquetadas;
- corregir vértebras fusionadas bajo un mismo ID;
- separar visualmente las regiones vertebrales cuando la máscara original no lo hacía;
- usar exclusivamente los colores definidos en la paleta;
- validar cada conversión con el script;
- comparar el número de etiquetas antes y después de la corrección.

En los casos donde se observaban 18 estructuras vertebrales, se decidió normalizar la máscara a 17 clases para mantener la consistencia del dataset. Esta decisión no pretende ser una validación clínica, sino una decisión técnica para mantener una convención uniforme en el proyecto.

Nota: En algunos casos se observaron hasta 18 estructuras visibles. Aunque anatómicamente pueden existir variaciones individuales, para este proyecto se decidió mantener la convención de 17 vértebras etiquetadas desde T1 hasta L5. Por esta razón, algunos casos fueron normalizados a 17 clases. Estas decisiones deben considerarse como parte de la curación técnica del dataset y pueden ser revisadas posteriormente por un experto clínico si se requiere mayor validación anatómica.

Se evidencio 4 casos así,

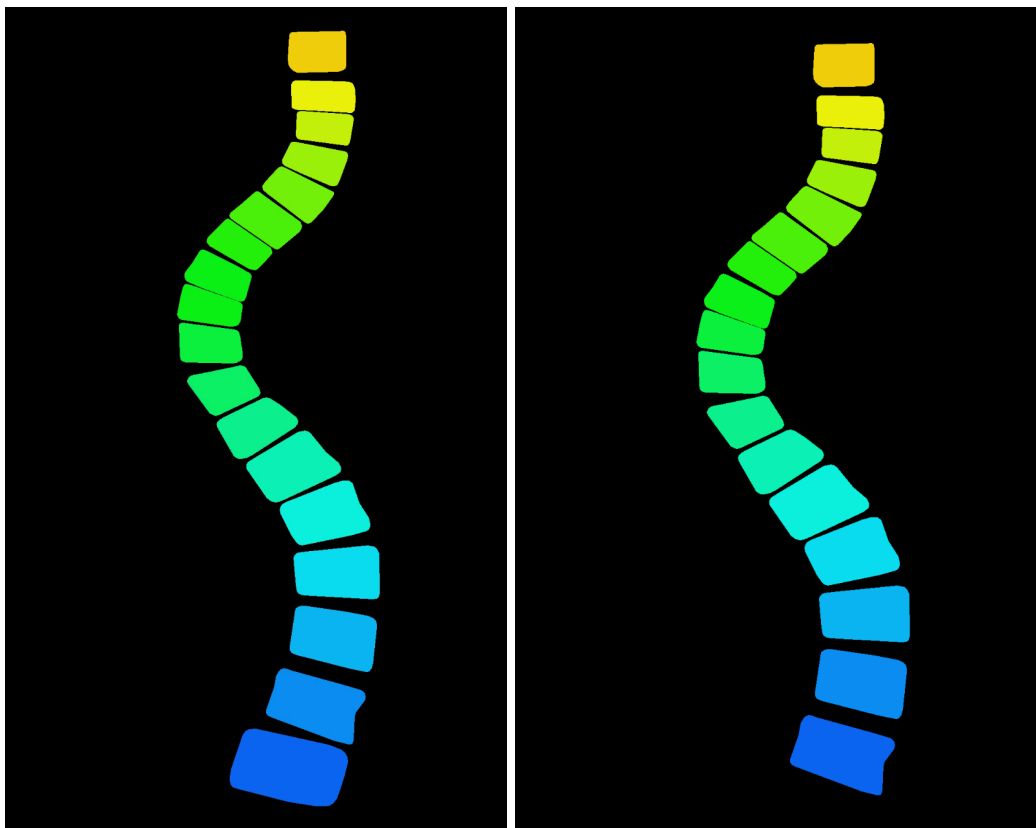
- **S_35**
- **S_93**
- **S_95**
- **S_97**

Como ejemplo se muestra el caso **S_97**. En esta imagen se identifican **17 etiquetas**, aunque visualmente se observan **18 estructuras vertebrales**. La imagen de la izquierda corresponde a la máscara original y la imagen de la derecha corresponde a la versión modificada.

En este caso existe incertidumbre anatómica: es posible que la estructura clasificada como **T1** corresponda realmente a **C7**, o que la última estructura visible corresponde a **S1** en lugar de **L5**. Sin embargo, para mantener el mismo criterio aplicado durante todo el proceso de corrección, se asumió que **T1 estaba correctamente identificada**. Bajo esta convención, se decidió excluir la última estructura visible de la máscara corregida y conservar únicamente las **17 clases correspondientes a T1–L5**.

En este caso existe incertidumbre anatómica, ya que la estructura adicional podría deberse a distintas situaciones: por ejemplo, una vértebra visible fuera del rango esperado **T1–L5 (C7/S1)**, una posible transición anatómica, una vértebra adicional. Casos como este también podrían implicar que la estructura marcada como **T1** no corresponda exactamente a T1, o que la última estructura visible no corresponda exactamente a L5. Sin embargo, al no contar con una validación clínica específica para este caso, se mantuvo el mismo criterio aplicado durante el proceso de corrección: asumir que **T1 estaba correctamente identificada**.

Esta decisión se tomó con fines de consistencia técnica del dataset y no debe interpretarse como una validación anatómica definitiva.



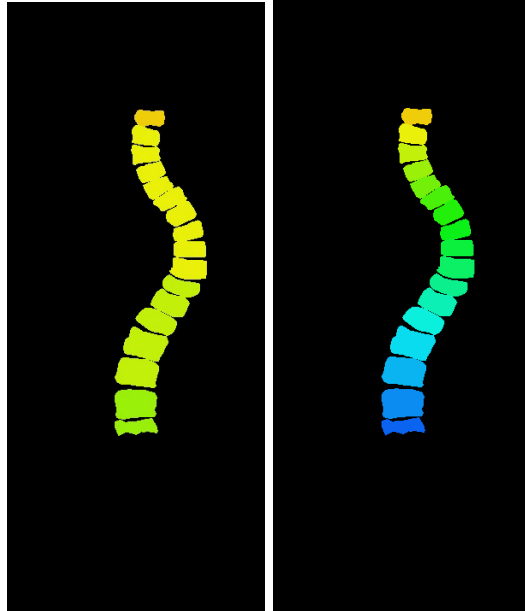
Izquierda: original - Derecha: Corrección

10. Ejemplos destacados

Caso S_156: corrección principal

El caso **S_156** fue uno de los ejemplos más representativos. Inicialmente el reporte indicaba solo 4 etiquetas correctamente diferenciadas, aunque visualmente se identificaban 17 vértebras.

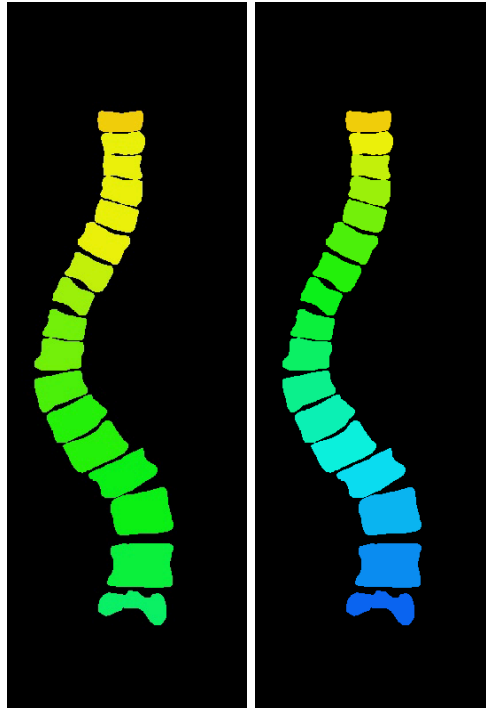
Después de la edición manual y la conversión, la máscara quedó con 17 etiquetas.



Caso S_180: mejora significativa

El caso **S_180** pasó de 10 etiquetas a 17.

Este caso muestra una corrección importante, aunque menos extrema que S_156.



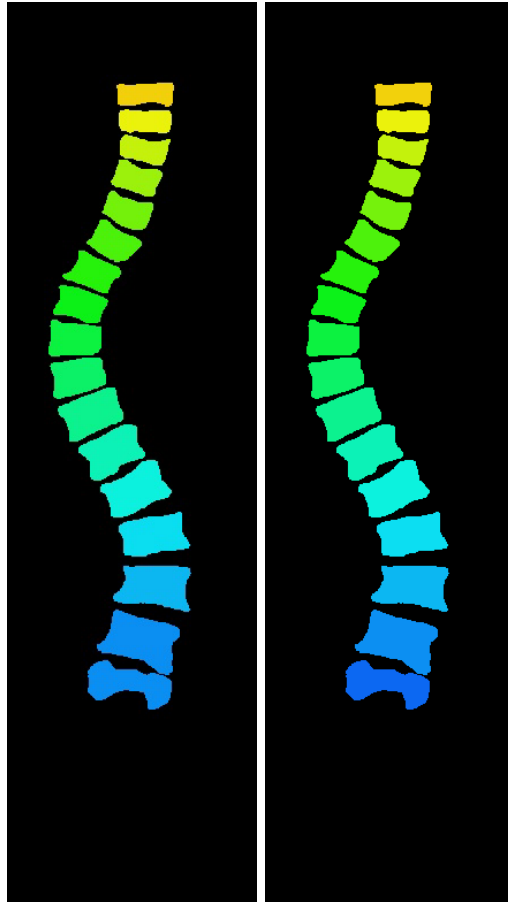
Caso S_146: diferencia mínima

El caso **S_146** pasó de 16 a 17 etiquetas. En este caso, la diferencia fue mínima y está asociada a una pequeña región donde 2 vértebras están “comparten un pixel”. [Ejemplo en Paso 3.](#)

Este ejemplo es útil para mostrar que algunos errores eran visualmente pequeños, pero relevantes para la consistencia de la máscara.

Caso S_202: última vértebra

El caso **S_202** pasó de 16 a 17. Visualmente, el problema parecía estar en la última vértebra donde se estaba clasificando las últimas 2 vértebras cómo la misma (mismo color/ L4), la corrección se evidencia al visualizar la parte inferior de la imagen donde se cambia la última vértebra para que tenga las 17 etiquetas (incluyendo la L5).



Casos con 18 estructuras visibles

Los siguientes casos fueron identificados como imágenes con 18 estructuras visibles:

- S_35
- S_93
- S_95
- S_97

Sin embargo, para mantener la consistencia del dataset, se normalizaron 17 etiquetas.

- S_35: 17 desde 14
- S_93: 17 desde 14
- S_95: 17 desde 16
- S_97: 17 desde 17 (tenia 17 etiquetas originalmente pero 18 vértebras)

Caso S_137: caso dudoso

El caso **S_137** fue marcado como posible caso a excluir o revisar con más cuidado.

Este caso parece no iniciar claramente en T1, por lo que no debería tratarse como una corrección estándar sin revisión adicional. En este caso lo que se corrige es 1 etiqueta donde originalmente tiene 13 pero con la corrección tiene 14, sin embargo este caso es diferente a los demás.

11. Resumen completo de máscaras corregidas

En total, esta lista manual contiene **147 máscaras revisadas/corregidas**:

- **8 casos N**
- **139 casos S**
- **115 casos terminaron con 17 etiquetas**
- **26 casos terminaron con 16 etiquetas**
- **4 casos terminaron con 15 etiquetas**
- **2 casos terminaron con 14 etiquetas**

La mayoría de los casos mejoraron respecto al conteo original de etiquetas. En conjunto, la lista pasa de **1680 etiquetas identificadas inicialmente** a **2459 etiquetas después de la corrección manual**, de acuerdo con el resumen manual.

Se adjunta todas las máscaras que se corrigieron. El formato incluye el nombre de la imagen, las etiquetas finales que obtuvo esa imagen y el número de etiquetas originales que tenía. Para dar un ejemplo **N25: 17 desde 15** significa que la imagen N_25 se corrigió para tener 17 etiquetas en total donde originalmente tenía 15.

```
N25: 17 desde 15
N26: 17 desde 15
N27: 17 desde 14
N28: 17 desde 15
N30: 16 desde 14
N31: 16 desde 11
N36: 17 desde 15
N65: 17 desde 15
```

```
S21: 17 desde 15
S22: 17 desde 10
S23: 17 desde 5
S24: 17 desde 10
S25: 17 desde 12
S26: 17 desde 4
S27: 17 desde 6
S28: 17 desde 8
S29: 17 desde 3
```



```
S30: 17 desde 10
S31: 17 desde 5
S32: 16 desde 12
S33: 17 desde 12
S34: 17 desde 10
S35: 17 desde 14 -- casó con 18 estructuras visibles
S36: 17 desde 12
S37: 17 desde 12
S39: 17 desde 4
S40: 17 desde 11
S41: 17 desde 7
S42: 17 desde 3
S43: 17 desde 13
S44: 17 desde 16
S45: 17 desde 15
S46: 17 desde 10
S47: 17 desde 6
S48: 17 desde 9
S49: 17 desde 4
S50: 17 desde 4
S51: 17 desde 5
S52: 17 desde 13
S53: 17 desde 13
S54: 17 desde 7
S55: 17 desde 8
S56: 17 desde 5
S57: 17 desde 9
S58: 17 desde 6
S59: 17 desde 14
S60: 14 desde 10
S61: 17 desde 12
S62: 17 desde 15
S63: 17 desde 5
S64: 17 desde 10
S65: 17 desde 8
S66: 17 desde 8
S67: 17 desde 12
S68: 17 desde 12
S69: 17 desde 8
S70: 17 desde 6
S71: 17 desde 9
S72: 17 desde 7
S73: 16 desde 11
```

```
S74: 17 desde 10
S75: 17 desde 11
S76: 17 desde 12
S77: 17 desde 12
S78: 17 desde 9
S79: 17 desde 13
S80: 17 desde 8
S81: 17 desde 4
S82: 17 desde 10
S83: 17 desde 7
S84: 17 desde 7
S85: 17 desde 13
S87: 17 desde 11
S88: 16 desde 9
S89: 17 desde 11
S90: 17 desde 12
S93: 17 desde 14 -- casó con 18 estructuras visibles
S95: 17 desde 16 -- casó con 18 estructuras visibles
S97: 17 desde 17 -- casó con 18 estructuras visibles
S102: 17 desde 16
S104: 17 desde 15
S105: 16 desde 15
S108: 17 desde 16
S110: 17 desde 16
S113: 17 desde 15
S118: 16 desde 14
S120: 16 desde 14
S121: 16 desde 15
S122: 17 desde 16
S124: 17 desde 16
S129: 17 desde 15
S131: 16 desde 15
S134: 15 desde 14
S137: 14 desde 13 -- caso dudoso / posible exclusión
S139: 17 desde 16
S140: 17 desde 16
S141: 15 desde 12
S145: 17 desde 14
S146: 17 desde 16 -- corrección mínima / diferencia de 1 píxel
S151: 16 desde 11
S152: 17 desde 11
S153: 17 desde 10
S154: 17 desde 11
```

```
S155: 16 desde 10
S156: 17 desde 4 -- ejemplo principal
S158: 17 desde 11
S160: 17 desde 15
S161: 16 desde 10
S162: 17 desde 10
S163: 16 desde 9
S164: 17 desde 13
S165: 17 desde 15
S166: 17 desde 11
S167: 16 desde 11
S168: 17 desde 10
S169: 17 desde 15
S170: 16 desde 12
S172: 17 desde 10
S173: 17 desde 11
S174: 15 desde 14
S175: 17 desde 12
S176: 17 desde 14
S178: 16 desde 11
S179: 17 desde 13
S180: 17 desde 10 -- ejemplo destacado
S181: 17 desde 12
S182: 17 desde 13
S183: 16 desde 10
S184: 16 desde 11
S185: 17 desde 16
S186: 17 desde 15
S188: 16 desde 11
S189: 17 desde 14
S190: 16 desde 14
S192: 17 desde 15
S193: 15 desde 14
S194: 16 desde 12
S195: 16 desde 12
S196: 16 desde 13
S197: 16 desde 14
S198: 17 desde 12
S201: 17 desde 14
S202: 17 desde 16 -- caso de última vértebra
S203: 17 desde 11
S204: 17 desde 13
S205: 17 desde 15
```

12. Limitaciones y consideraciones

Este proceso fue una corrección técnica y visual del dataset. Aunque se realizó una revisión cuidadosa caso por caso, no debe interpretarse como una validación clínica definitiva.

Las principales limitaciones son:

- algunas decisiones se tomaron visualmente;
- algunos casos pueden requerir revisión anatómica experta;
- en imágenes con 18 estructuras visibles se prioriza la consistencia del dataset;
- algunos casos quedaron con menos de 17 etiquetas

Se quiere aclarar que solamente se hizo la corrección sobre las máscaras que ya existían, no en este proceso no se intentó agregar manualmente etiquetas a vértebras nueva o no identificadas (casos que tenían menos de 17 vértebras)

Aun así, el proceso mejora de forma importante la consistencia de las máscaras respecto al objetivo del proyecto: segmentar vértebras desde T1 hasta L5 y también se mejoró el rendimiento de los modelos.

13. Conclusión de correcciones de máscaras

La revisión permitió identificar múltiples máscaras en las que las vértebras estaban presentes visualmente, pero no correctamente separadas en términos de etiquetas. Mediante un flujo de conversión ID $\rightarrow$ RGB, edición manual en GIMP, validación estricta de colores y reconversión RGB $\rightarrow$ ID, se generaron máscaras corregidas más consistentes para el entrenamiento/evaluación de modelos de segmentación.

El proceso dejó evidencia en tres niveles:

1. Máscaras RGB editadas,
2. Máscaras ID corregidas,
3. Archivo de logs con comparación entre IDs originales y corregidos.

Generación de pre máscaras binarias para expansión del dataset

Después de corregir las máscaras multicategoría originales, se inició una segunda fase del proyecto orientada a la **expansión del dataset**. En esta etapa, el objetivo no es identificar cada vértebra individualmente como T1, T2, T3, etc., sino generar una máscara binaria preliminar que separe la región vertebral del fondo.

La convención utilizada para estas máscaras binarias es:

```
0 = fondo  
255 = región de columna / vértebras
```

Este enfoque permite crear una primera aproximación automática de la zona de interés. Posteriormente, estas pre máscaras pueden ser revisadas y corregidas manualmente en GIMP, de forma similar al flujo utilizado en la corrección de máscaras multicategoría.

La idea principal es reducir el trabajo manual desde cero: en lugar de dibujar toda la máscara manualmente, se parte de una predicción generada por el modelo y luego se corrige visualmente.

1. Modelo binario utilizado

Para esta fase se entrenó un modelo de segmentación binaria basado en **DeepLabV3+**. A diferencia del modelo multicategoría, este modelo no intenta distinguir entre T1, T2, T3 o L5. Su función es detectar la región general de la columna o de las vértebras frente al fondo.

El modelo fue entrenado usando las imágenes y máscaras disponibles del dataset original. Las máscaras multicategoría, incluyendo las corregidas manualmente cuando estaban disponibles, fueron convertidas a formato binario. Es decir, cualquier píxel con un ID mayor que cero fue considerado como parte de la región vertebral.

Conceptualmente, la conversión fue:

```
binary_mask = (id_mask > 0)
```

De esta manera, todas las clases vertebrales T1–L5 se agrupan en una sola clase positiva.

El modelo obtuvo resultados cercanos a un **Dice de 0.90**, lo cual indica que puede ser útil como herramienta de generación de pre máscaras. Sin embargo, estas predicciones no se consideran etiquetas finales. Su propósito es servir como punto de partida para revisión y corrección manual.

2. Fuente de nuevas imágenes

Para ampliar la cantidad de imágenes disponibles, se utilizó un dataset adicional proveniente de **Roboflow Universe**, llamado **scoliosis2**. La página del dataset lo presenta como un dataset de detección de objetos con clases como **normal spine**, **scoliosis spine**, **vertebra** y variantes similares.

Link: <https://universe.roboflow.com/college-wtsl5/scoliosis2-dvnfp>

Aunque el dataset fue descargado en formato YOLOv8 y contiene anotaciones, estas anotaciones no fueron utilizadas para entrenar el modelo de segmentación binaria. Para este proyecto, el uso principal del dataset fue obtener **imágenes crudas adicionales** de radiografías de columna y aprovechar la clasificación general para separar imágenes normales e imágenes con escoliosis.

3. Limpieza y selección del nuevo dataset

Antes de usar estas imágenes como base para nuevas pre máscaras, se realizó un proceso de limpieza y selección. El objetivo fue conservar únicamente imágenes útiles para el flujo de segmentación y descartar aquellas que podían introducir ruido o errores.

El proceso general fue el siguiente:

1. Descargar el dataset adicional desde Roboflow.
2. Extraer las imágenes crudas.
3. Separar imágenes normales e imágenes con escoliosis.
4. Ignorar las anotaciones originales para el entrenamiento de segmentación.
5. Eliminar imágenes mezcladas, ambiguas o poco útiles.
6. Revisar manualmente imágenes de baja calidad o de poco uso práctico para nuestro proyecto.
7. Eliminar imágenes repetidas.
8. Revisar posibles duplicados frente al dataset original.
9. Revisar duplicados internos y posibles imágenes volteadas/espejadas.
10. Renombrar las imágenes seleccionadas siguiendo la convención del proyecto.
11. Crear manifiestos para mantener la trazabilidad entre nombres originales y nombres nuevos.

Este proceso permitió construir un subconjunto más limpio y coherente con el objetivo del proyecto.

4. Dataset final listo para etiquetado

Después de la limpieza, se obtuvo un conjunto final de imágenes listas para generar pre máscaras y posteriormente corregirlas manualmente.

La carpeta principal es:

```
scoliosis_dataset_extra_roboflow_v16_final_selection/
```

Dentro de esta carpeta, el conjunto principal se encuentra en:

```
ready_for_labeling/  
  normal/  
  scoliosis/
```

El conteo actual es:

```
normal:      44 imágenes  
scoliosis:   219 imágenes  
total:       263 imágenes
```

Estas 263 imágenes fueron consideradas útiles para continuar el flujo de generación de pre máscaras binarias.

También existe una carpeta adicional:

```
future_review_candidates/  
  scoliosis/
```

Esta carpeta contiene aproximadamente:

```
scoliosis: 631 imágenes
```

Estas imágenes no fueron incluidas todavía en el conjunto principal porque requieren una revisión posterior. Pueden ser útiles en una etapa futura, pero no forman parte del lote actual listo para etiquetado.

5. Convención de nombres

Para mantener consistencia con el dataset original, las imágenes seleccionadas fueron renombradas siguiendo la misma convención:

```
N = columna normal  
S = columna con escoliosis
```

El rango de nombres asignado fue:

```
Normal:  
N_072 a N_115  
  
Escoliosis:  
S_207 a S_425
```

Esto permite integrar las nuevas imágenes con el flujo ya existente sin romper la organización del proyecto.

La relación entre el nombre original del archivo y el nuevo nombre asignado se conserva en:

```
manifests/ready_for_labeling_manifest.csv
```

Este archivo es importante porque permite mantener la trazabilidad del origen de cada imagen.

6. Generación de pre máscaras binarias

Con el modelo binario entrenado, se generaron predicciones para las imágenes nuevas del conjunto **ready_for_labeling**.

El espacio de trabajo utilizado para esta fase es:

```
spine_pred/
```

La estructura esperada es:

```
spine_pred/  
  models/  
    best_model.pth  
  
  scripts/  
    predict_ready_images.py  
  
  outputs/  
    overlays/  
    masks_png/
```



```
probability_maps/  
manifests/
```

El modelo guardado en:

```
models/best_model.pth
```

corresponde al mejor modelo obtenido durante el entrenamiento binario.

El script:

```
scripts/predict_ready_images.py
```

Se utiliza para procesar las imágenes nuevas y generar las salidas correspondientes.

7. Salidas generadas

El proceso de predicción genera principalmente tres tipos de salida:

```
outputs/overlays/
```

Contiene imágenes donde la predicción aparece superpuesta sobre la radiografía original. Estas imágenes son útiles para una inspección visual rápida, ya que permiten revisar si el modelo está detectando correctamente la región de la columna.

```
outputs/masks_png/
```

Contiene las máscaras binarias generadas por el modelo. Estas son las imágenes que posteriormente pueden abrirse en GIMP para corrección manual.

```
outputs/probability_maps/
```

Contiene mapas de probabilidad generados por el modelo. Estos pueden ser útiles para análisis técnico, ya que muestran qué tan seguro estaba el modelo en distintas regiones de la imagen.

También se generan manifiestos en:

```
outputs/manifests/
```

Estos archivos ayudan a mantener control sobre qué imágenes fueron procesadas y qué salidas fueron generadas.

8. GUIA paso por paso

Configuraciones Básicas

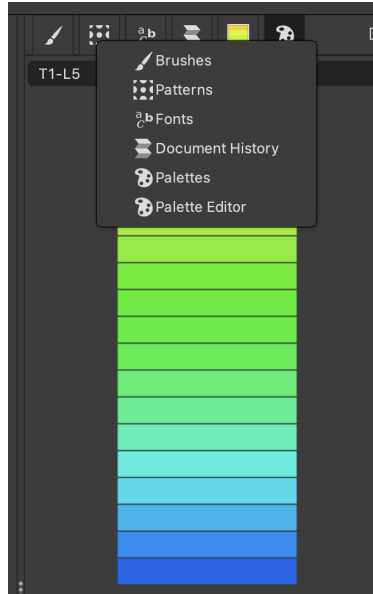
Descargar GIMP o usar un software que nos permita modificar imágenes.

Link: <https://www.gimp.org/>

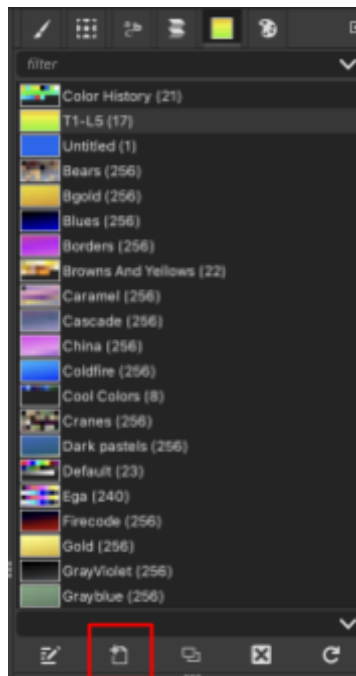
Para configurar GIMP se sugiere crear una paleta de colores, en la ventana de la derecha se puede agregar una.



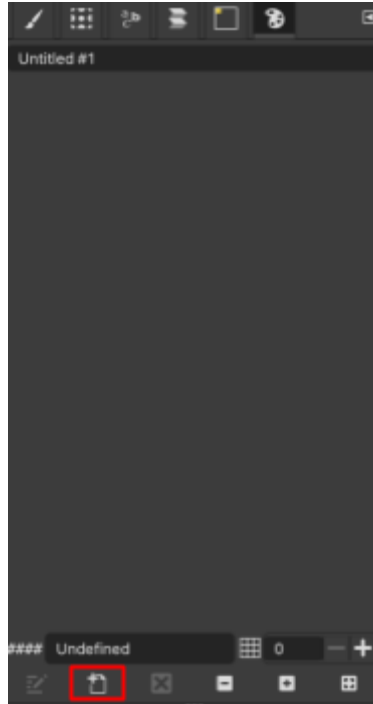
Estos colores nos van ayudar a distinguir las diferentes vértebras. Aunque hay una sugerencia de colores, estos se pueden cambiar. (Nota Importante: si se cambian en la paleta de colores también se tienen que cambiar en el script de Python)



Al darle clic derecho se selecciona Palettes



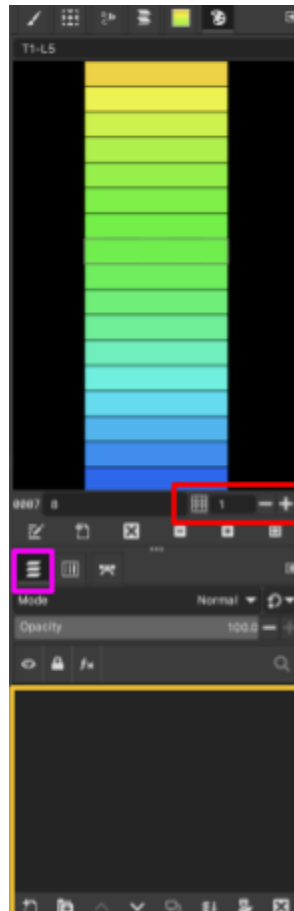
Se selecciona el botón para agregar un grupo nuevo de colores.



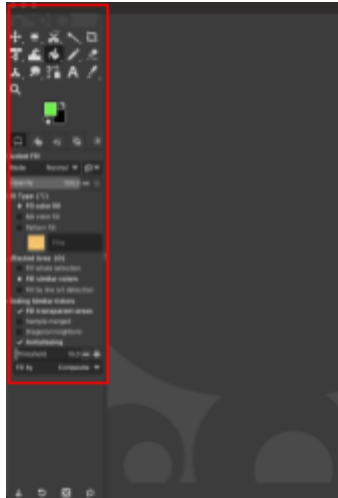
Con el mismo botón se pueden agregar más colores, también se puede cambiar el nombre de cada color para identificarlo más fácil. Los colores que se recomienda tener son los siguientes 17, cada uno representa una vértebra desde la vértebra T1 hasta la L5. Como se mencionó anteriormente, se pueden cambiar los colores teniendo en cuenta que en el script de python también se deben cambiar (Nota: esto puede ser útil para las parte de la vértebra T6,T7,T8 y T9 donde hay colores verdes similares)

```
1: "#F2D10C",
2: "#EBF20C",
3: "#C4F20C",
4: "#9CF20C",
5: "#75F20C",
6: "#4DF20C",
7: "#26F20C",
8: "#0CF219",
9: "#0CF240",
10: "#0CF268",
11: "#0CF28F",
12: "#0CF2B7",
13: "#0CF2DE",
14: "#0CDEF2",
15: "#0CB7F2",
16: "#0C8FF2",
17: "#0C68F2",
```

Esto debe quedar similar a la siguiente imagen, la parte en rojo representa la cantidad de columnas, en este caso se recomienda dejarlo en 1, pero se puede cambiar dependiendo de su preferencia. En fucsia se muestran las capas “layers” donde se harán las modificaciones de los elementos importados. En amarillo van a quedar las imagenes que se importan, es importante tener seleccionada la capa que se quiere editar/modificar. (Nota: Se tiene que hacer clic sobre cualquier color para cambiarlo)

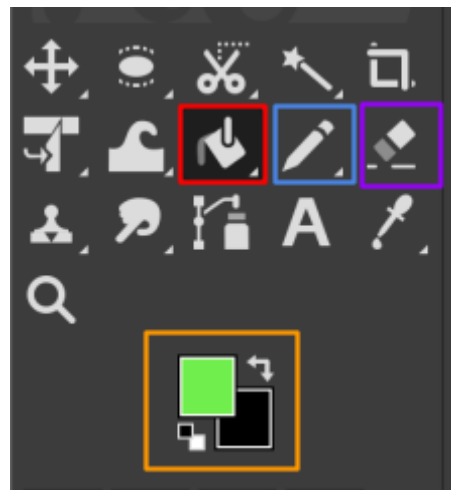


A la izquierda, tenemos las herramientas básicas que nos permiten hacer estas modificaciones, y poder etiquetar bien las vértebras.



En particular hay 3 herramientas que nos van a servir mucho para hacer estas modificaciones.

- La primera “Bucket Fill Tool” en rojo nos va ayudar a rellenar de secciones grandes.
- La segunda “Pencil Tool” en azul es nuestra herramienta principal para agregar porciones de las vértebras y poder cambiar el color de cada vértebra.
- La tercera “Eraser Tool” en morado es para quitar porciones de la máscara que no fueron generadas correctamente.
- La cuarta en naranja es la herramienta que usamos para saber qué color tenemos seleccionado en ese momento.



Notas importantes de configuración.

- Las configuraciones del Pencil Tool, tener tener la *opacidad* al 100.0 y *hardness* al 100.0. El *Size* se puede modificar dependiendo de las regiones que se quieran trabajar, por lo general usar 1.00 pixeles es bueno para tener mayor precisión.

- Para el Eraser Tool, se debe tener una *opacidad* de 100.0, *Hardness* 100.0 y tener el checkbox de *Hard edge* seleccionado.
- Para Bucket Fill Tool, los que tiene por defecto están bien, asegurarse que el Threshold esté en 0.0.

Todas estas configuraciones están justamente debajo.

Con respecto a los datos, se pueden encontrar en la carpeta de drive:

https://drive.google.com/drive/folders/1KdM9arg5JxzSYuNe4oRRGNvY10hu_dEI?usp=sharing

Aquí se encuentran la carpeta “vetebra_mask_correction” la que contiene las correcciones previamente exploradas. Este documento, el script de python que se usa llamado “mask_correction_workflow.py” que en este caso nos va ayudar a convertir nuestras imágenes a las máscaras deseadas. (se explorará un el script de python más adelante)

En la carpeta “Nuevos Datos” se encuentran lo datos que vamos a utilizar para este proceso, esto contiene:

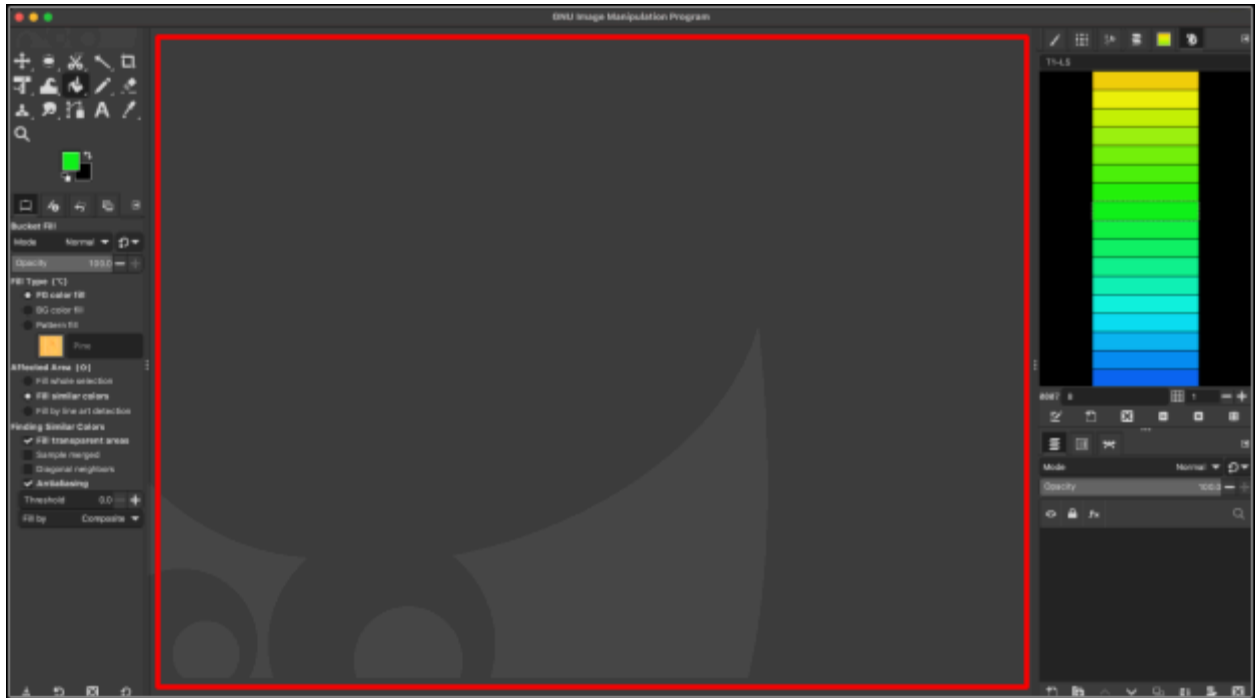
Uno zip que se llama **outputs.zip**, al descomprimirlo esto tendrá las siguiente carpetas:

- Overlays: muestra las imágenes originales con el overlay de la máscara que se generó, esto nos ayuda identificar los candidatos más fáciles para trabajar.
 - Hay 2 carpetas que representan las imágenes, las columnas que son normales y las que tienen escoliosis
- manifest : Se puede ignorar
- masks_png: contiene las imágenes en formato png que representan las máscaras binarias de la columna.
 - Hay 2 carpetas que representan las imágenes, las columnas que son normales y las que tienen escoliosis
- probability_maps: Se puede ignorar

Otro zip que se llama **ready_for_labeling.zip**, estas son las imágenes originales que vamos a utilizar. Al descomprimir este zip tendremos dos carpetas, las que son normales y las que tienen escoliosis.

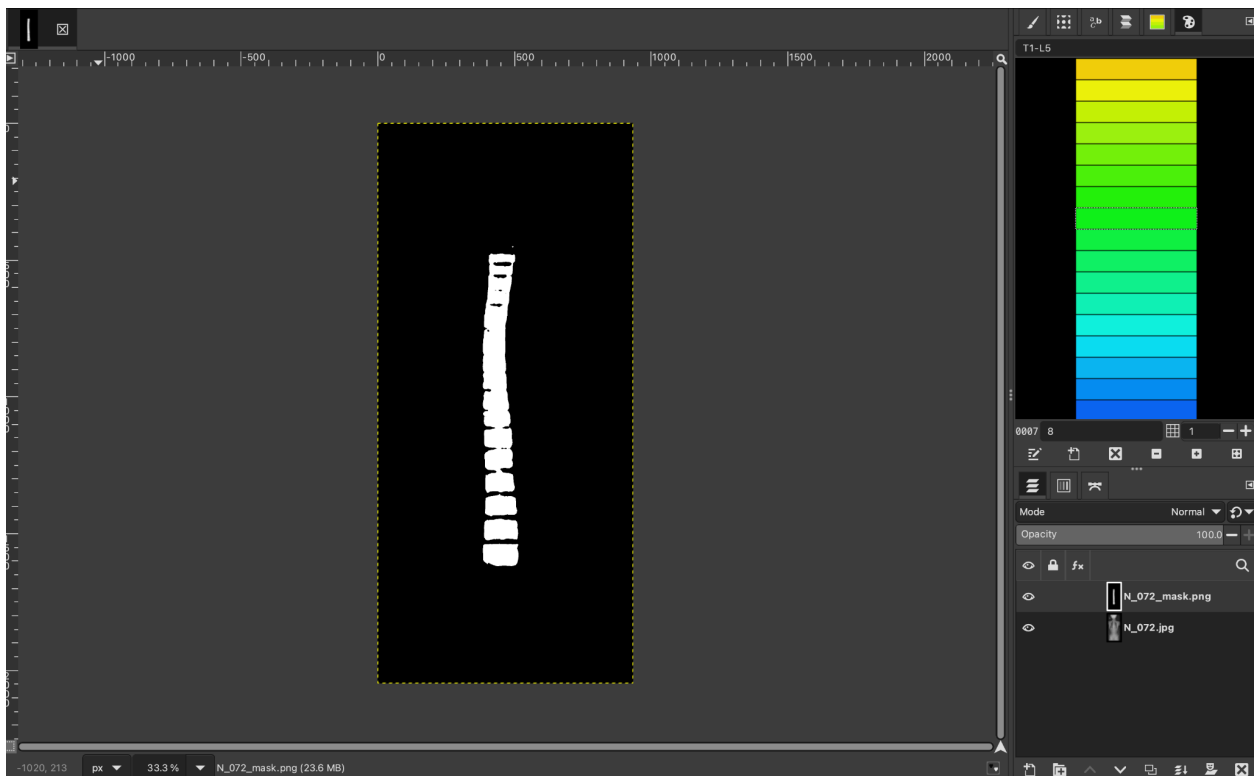
Paso 1

Arrastrar/importar 2 imágenes, la primera es la imagen original que se encuentra en “ready_for_labeling”, la segunda es la máscara correspondiente a la imagen encontrada en “outputs/masks_png”.

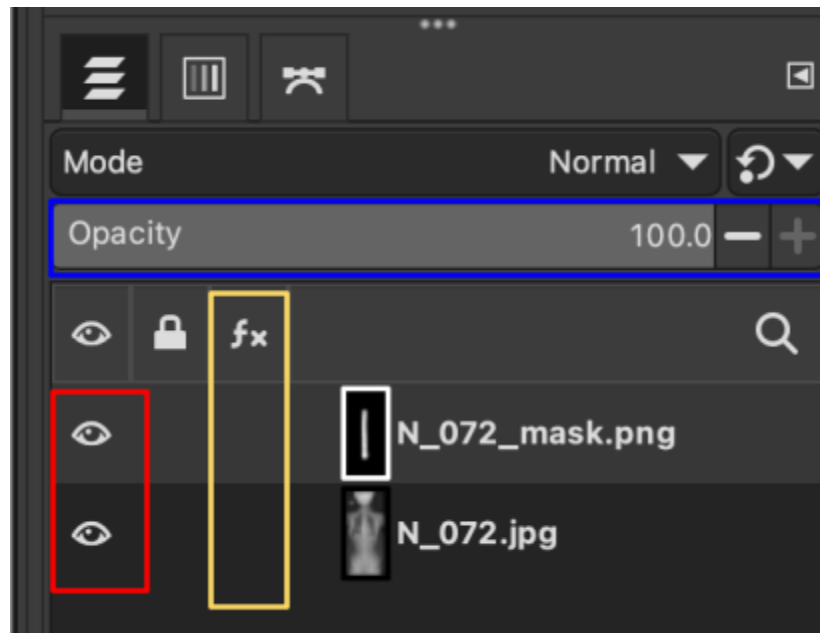


Paso 2

Al importar las 2 imágenes deberíamos tener una vista similar a la siguiente.

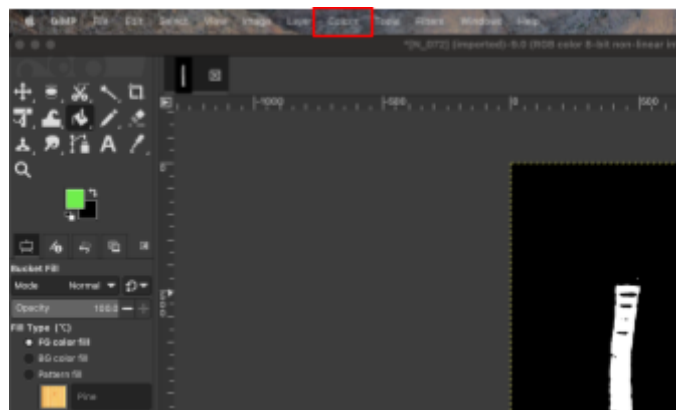


Se puede confirmar que el nombre de la máscara y el nombre de la imagen son los mismos. Adicionalmente es necesario tener la máscara encima de la imagen original, es decir debe ser la primera capa. En rojo se puede mostrar o esconder la capa, en amarillo se podrán encontrar todos los efectos que se aplican a la capa, esto es muy importante para un proceso mas adelante, en esta referencia ninguna de las imagenes tiene efectos. Y por último en azul tenemos la opacidad de la capa, al seleccionar una capa se podrá modificar la opacidad para poder visualizar cambios (esto nos sirve principalmente para la capa con la máscara).

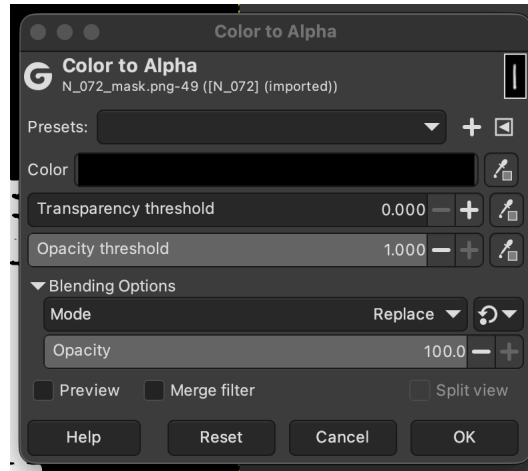


Paso 3

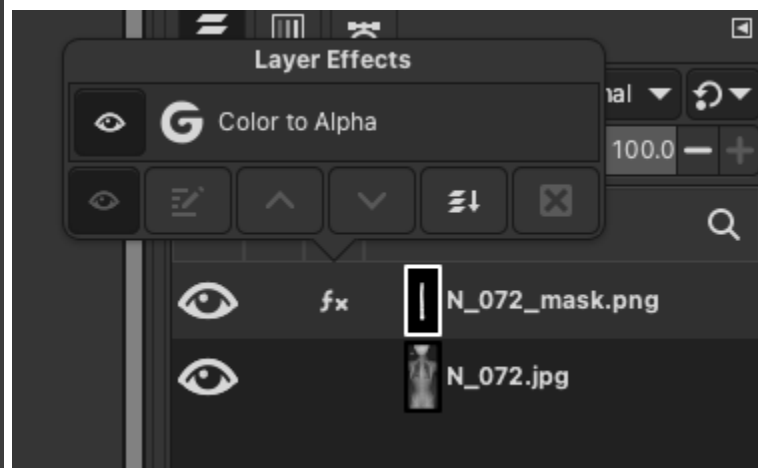
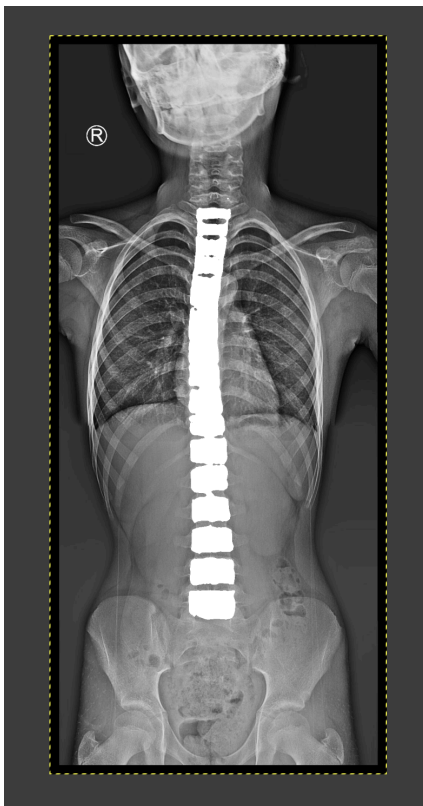
Con la capa de la máscara escogida tenemos que quitar el “fondo” negro para poder hacer modificaciones. Para esto vamos a ir a la opciones “Colors” después “Color to Alpha”.



Verifica que el color sea negro y dale clic en Ok.



Lo que nos debería mostrar lo siguiente, en la mitad nos muestra la imagen y la máscara en blanco, utilizando “Bucket Fill Tool” se puede cambiar el color de la columna para poder identificarla un poco más fácil. En la parte de las capas, se podrá ver el efecto que le agregamos a la máscara, si hacemos clic en el ojo quitamos esos efectos.



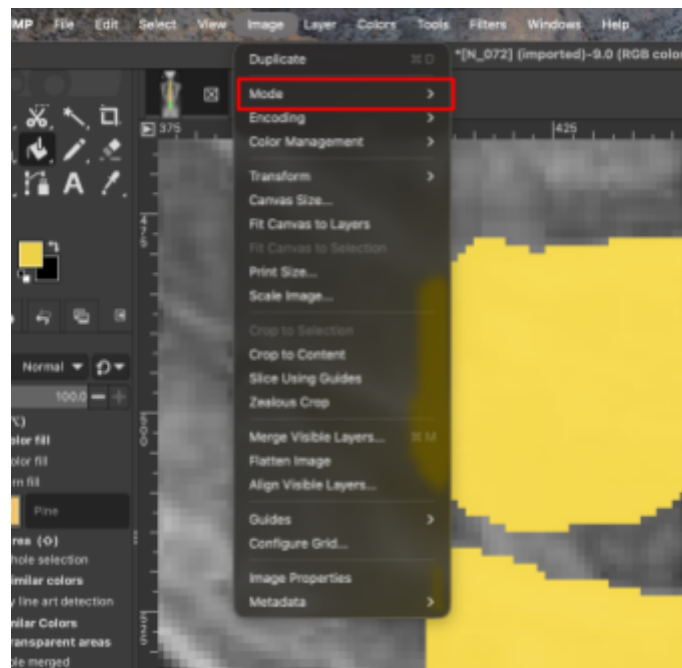
Paso 4

Ahora podemos realizar las modificaciones, en este caso nos importa simplemente separar y corregir la máscara creada, borrando porciones equivocadas y agregando pedazos donde se

necesitan para cubrir la columna completa. NOTA: El objetivo principal en este paso es simplemente corregir la máscara binaria.

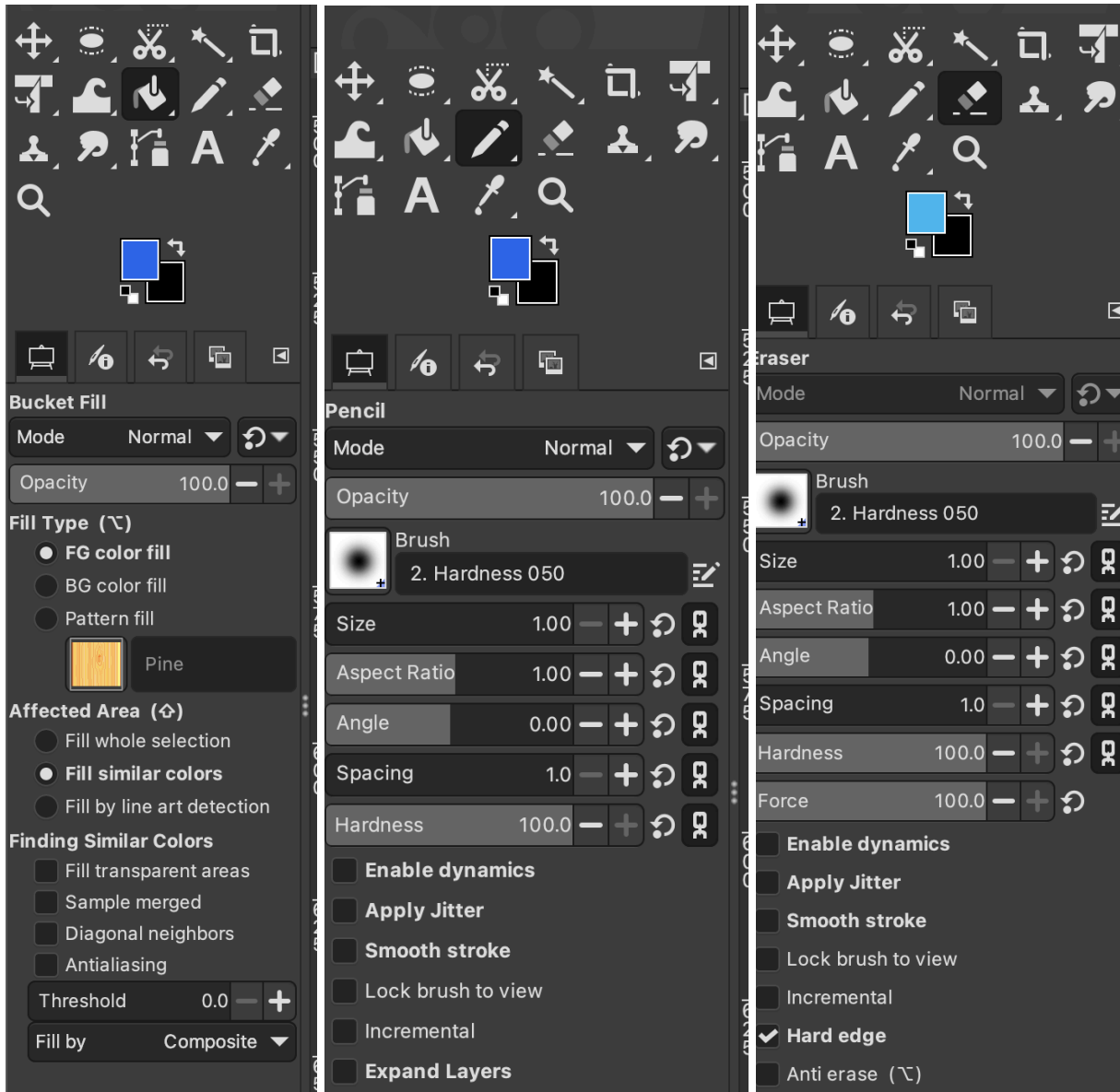
NOTA 2: Por lo general el modelo es muy bueno identificando la T1 (la columna que necesitamos con 17 vértebras), es decir identifica muy bien la primera vértebra T1 hasta la L5, aunque puede tener errores en la máscara, es un acercamiento bueno. También es importante contar 17 vértebras desde la primera, para asegurarnos que estamos en el camino correcto y guiarnos de la radiografía para hacer correcciones si son necesarias.

En este paso también es importante asegurarnos que la imagen tenga el modo de colores RGB. Ir a Images → Mode → RGB



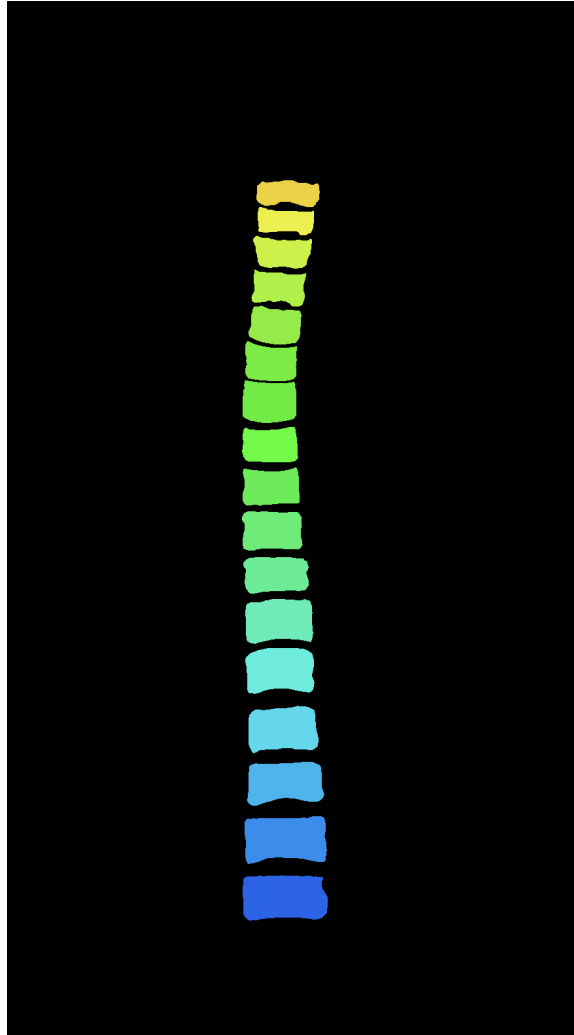
Paso 5

Verifica que las siguientes herramientas tengan las configuraciones similares.



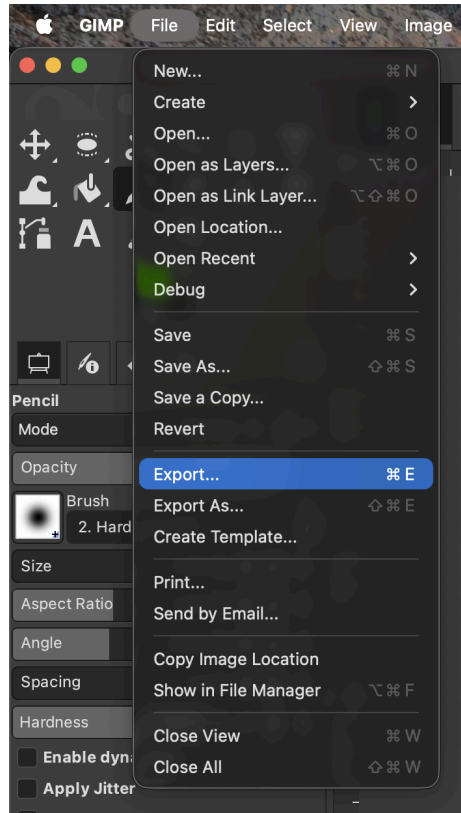
Es muy importante para este paso esconder el efecto de la mascara, al hacer esto, deberia mostrar el fondo negro de nuevo y la columan solamente. La imagen original no estará visible. (Revisar que esta capa tenga opacidad de 100.0)

Ahora con la máscara de la columna correctamente segmentada, se podrá agregar colores. Estos colores se usan para poder identificar la vértebras de manera más fácil y después convertir en las máscaras con 18 etiquetas. Para esta “colorea” cada vértebra con un nuevo color comenzando desde 1 hasta 17.

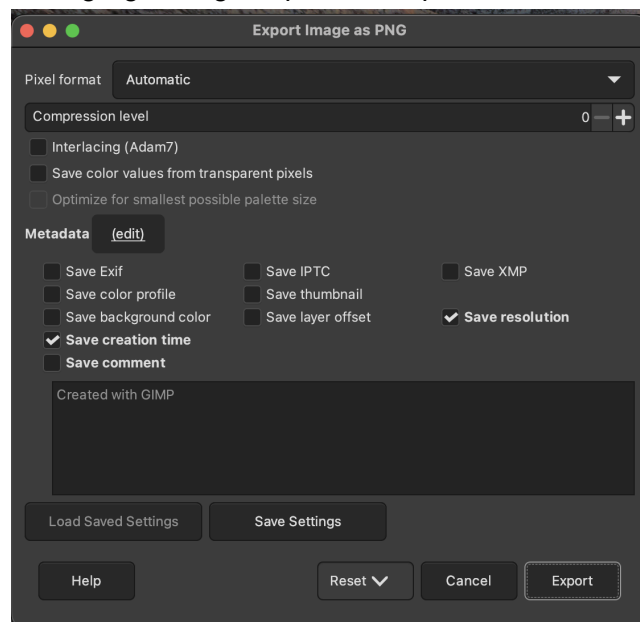


Paso 6

Al tener la imagen con los diferentes colores se va a exportar como png. Esta la podemos guardar en la carpeta Output, creando una nueva carpeta llamada "LabelMultiClass_Color_EDIT" en donde tendremos todas nuestras nuevas máscaras.



Al guardarla NO queremos agregar ningún tipo de compresión.



Paso 7

Nota importante sobre el uso del código:

El código incluido en esta sección debe entenderse como una guía práctica del flujo utilizado, no como una plantilla rígida que pueda ejecutarse sin revisar rutas, nombres de carpetas y archivos disponibles en cada entorno. Antes de ejecutar el proceso, se debe verificar que la imagen original, la máscara binaria generada y las carpetas de salida estén ubicadas correctamente. En particular, para la revisión manual se requiere tener disponible la imagen original y su máscara correspondiente, ya sea desde `ready_for_labeling/` y `outputs/masks_png/` en el caso de las pre máscaras binarias, o desde `LabelMultiClass_Color_EDIT/` y `LabelMultiClass_ID_PNG_corrected/` en el caso de las máscaras multicategoría corregidas. Las rutas del script pueden necesitar ajustes menores según la estructura local del proyecto.

Al tener la máscara guardada, se tendrá que hacer algunas modificaciones al script de python. Principalmente en las rutas principales, para eso encuentra la ubicación local de estos archivos y reemplaza los siguientes:

```
PROJECT_ROOT: Path =  
Path("/Users/jorgeonate/Desktop/programming/Scoliosis_Dataset/vertebra_mask_correction")  
RADIOGRAPH_DIR: Path =  
Path("/Users/jorgeonate/Desktop/programming/Scoliosis_Dataset/Normal")  
ID_MASK_DIR: Path =  
Path("/Users/jorgeonate/Desktop/programming/Scoliosis_Dataset/LabelMultiClass_ID_PNG")
```

Por (teniendo en cuenta que estamos trabajando con imagenes “Normales” y con “Escoliosis”. Es decir cambiar el “normal” por “scoliosis” cuando sea necesario)

```
PROJECT_ROOT: Path = Path("Tu RUTA")  
  
# Para este flujo no necesitamos radiografía original.  
# Se deja como referencia, pero no se usará si solo corres convert_edited_color_to_id().  
RADIOGRAPH_DIR: Path = PROJECT_ROOT / "outputs" / "masks_png" / "normal"  
  
# Carpeta donde está la máscara RGB editada con los 17 colores.  
ID_MASK_DIR: Path = PROJECT_ROOT / "outputs" / "masks_png" / "normal"
```

OPCIONAL: Después tenemos que cambiar otras líneas para mantener una estructura similar, aunque no es necesario, una opcion. En nuestro caso no se hace el cambio, se crea una carpeta llamada `LabelMulticlass_Color_EDIT` para poder guardar las imagenes aqui.

```
COLOR_EDIT_DIR: Path = PROJECT_ROOT / "outputs" / "LabelMultiClass_Color_EDIT"  
CORRECTED_ID_DIR: Path = PROJECT_ROOT / "outputs" / "LabelMultiClass_ID_PNG_corrected"  
LOG_DIR: Path = PROJECT_ROOT / "outputs" / "logs"
```

NOTA IMPORTANTE: Se recomienda mantener esta ruta dado que es fácil de ubicar los archivos nuevos.

Ejemplo de otras rutas, es muy importante entender las rutas y el código antes de hacer cambios como estos.

```
COLOR_EDIT_DIR: Path = PROJECT_ROOT / "outputs" / "masks_png" / "normal"
CORRECTED_ID_DIR: Path = PROJECT_ROOT / "outputs" / "masks_id_png" / "normal"
LOG_DIR: Path = PROJECT_ROOT / "outputs" / "logs"
```

Se recomienda siempre revisar muy bien las ubicaciones que se guardan.

También se cambia esta función para que se ajuste a los cambios que hicimos, en este caso estamos quitando `_COLOR_EDIT` dado que nuestros archivos no se llaman así, sin embargo se puede mantener el código igual, solo se necesita guarda la imagen con el `"_COLOR_EDIT"` al final.

```
def editable_color_mask_path(base_name: str) -> Path:
    return CFG.COLOR_EDIT_DIR / f"{base_name}_COLOR_EDIT.png"
```

Se cambia a

```
def editable_color_mask_path(base_name: str) -> Path:
    return CFG.COLOR_EDIT_DIR / f"{base_name}.png"
```

Después abre un terminal, corre el script en la ruta donde tienes guardado el script

```
cd tu_ruta
```

Después corre este mismo script

```
python3 -i tu_ruta/mask_correction_workflow.py
```

Para salir de la consola se puede hacer con Ctrl + D o

```
exit()
```

Para hacer la conversión de esta imagen a las máscaras que necesitamos se tiene que hacer el comando. (Aquí va el nombre la las máscara tal cual como la guardaste, sin la extensión)

```
convert_edited_color_to_id("N_072_mask")
```


NOTA: En el caso que no funcione, agrega esta funcion justamente antes de main, esta nos ayudará identificar claramente dónde está el error

```
def show_bad_colors_red(base_name):
    rgb = load_edited_color_mask(editable_color_mask_path(base_name))

    bad_mask = np.zeros(rgb.shape[:2], dtype=bool)

    flat = rgb.reshape(-1, 3)
    colors, counts = np.unique(flat, axis=0, return_counts=True)

    print("Unexpected colors:")
    for color, count in zip(colors, counts):
        color_tuple = tuple(int(x) for x in color.tolist())
        if color_tuple not in RGB_TO_ID:
            print(f" RGB{color_tuple} -> {int(count)} pixels")
            bad_mask |= np.all(rgb == np.array(color_tuple, dtype=np.uint8), axis=-1)

    preview = rgb.copy()
    preview[bad_mask] = np.array([255, 0, 0], dtype=np.uint8)

    out_path = CFG.LOG_DIR / f"{base_name}_BAD_COLORS_RED.png"
    CFG.LOG_DIR.mkdir(parents=True, exist_ok=True)
    Image.fromarray(preview).save(out_path, format="PNG", compress_level=0)

    print(f"\nSaved preview with bad pixels in red:")
    print(out_path)

    plt.figure(figsize=(8, 14))
    plt.imshow(preview, interpolation="nearest")
    plt.title(f"Bad colors highlighted in red: {base_name}")
    plt.axis("off")
    plt.show()
```

Después corre este comando para ver esa imagen. Esto nos muestra en rojo las parte que tiene un color diferente a los requeridos.

```
show_bad_colors_red("N_072_mask")
```

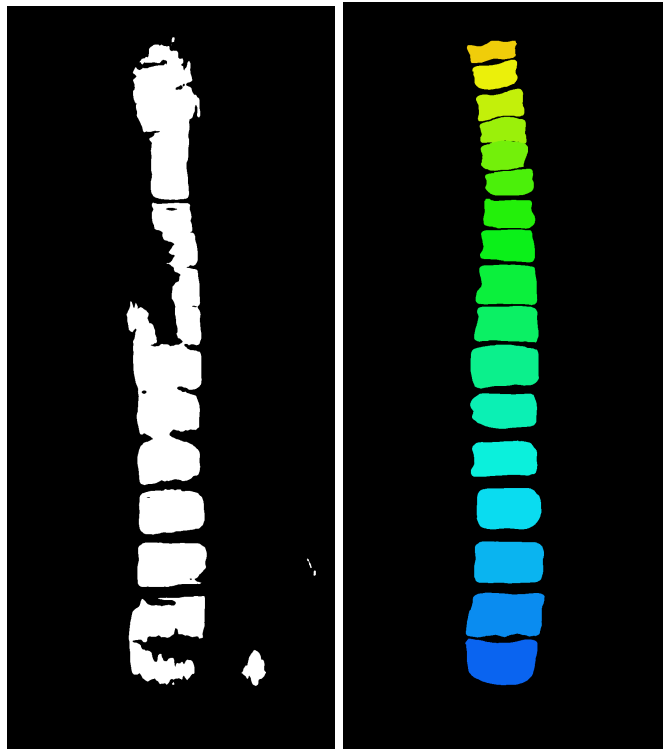
Como recomendación las imágenes se puede guardar simplemente con el número, en este caso será N_72.png

Al completar eso, debería mostrarnos en la consola algo como lo siguiente, aquí verificamos que hay 18 etiquetas desde el 0 hasta el 17, y podemos ir a la ubicación para confirmar que se creó la máscara correctamente.

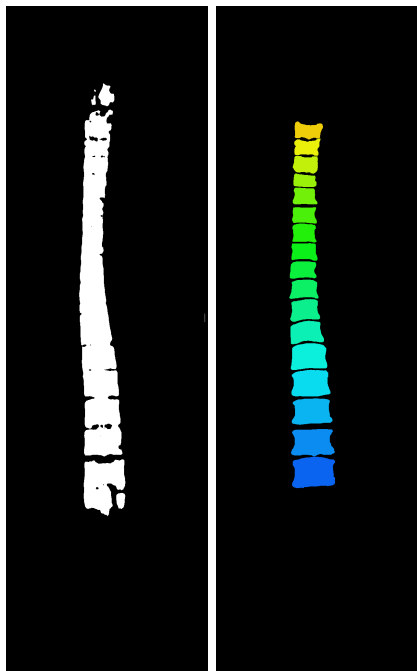
```
>>> convert_edited_color_to_id("N_072_mask")
Saved corrected ID PNG: /Users/jorgeonate/Desktop/programming/spine_pred/outputs/LabelMultiClass_ID_PNG_corrected/N_072_mask.png
Original unique IDs: [0, 12, 13, 38, 77, 117, 156, 196, 235, 242]
Corrected unique IDs: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17]
Correction applied: True
PosixPath('/Users/jorgeonate/Desktop/programming/spine_pred/outputs/LabelMultiClass_ID_PNG_corrected/N_072_mask.png')
```

Ejemplos de resultados

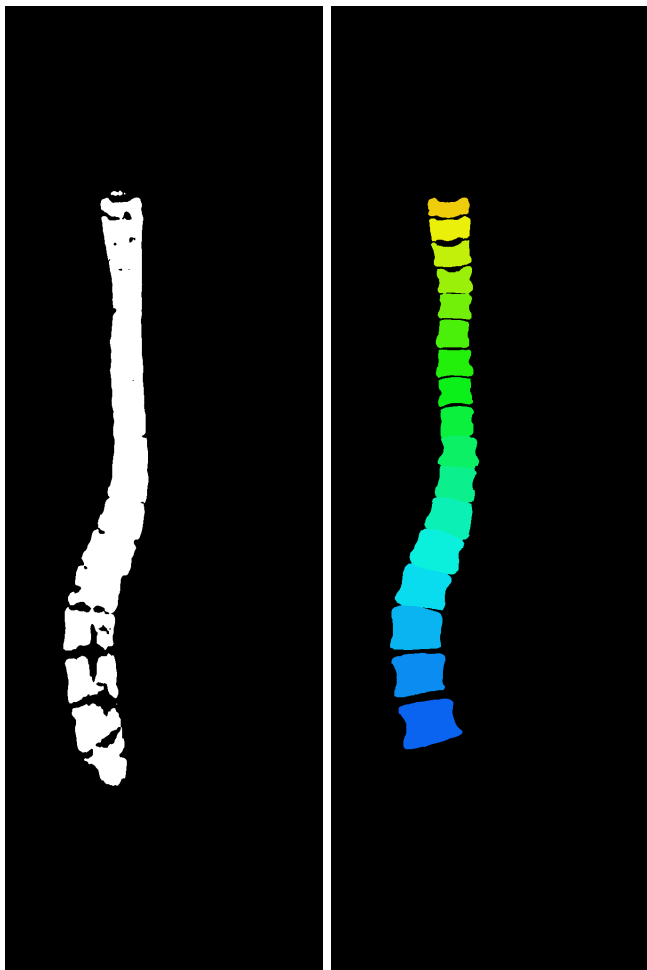
N_73



N_75



S_208



Las modificaciones realizadas en este proceso deben entenderse como una fase de curación técnica y preparación del dataset, no como una validación clínica definitiva. Las correcciones multicategorías se realizaron para mejorar la consistencia de las máscaras T1–L5 existentes, especialmente en casos donde las vértebras estaban presentes visualmente pero fusionadas o mal diferenciadas por ID.

En el caso de las nuevas pre máscaras binarias, el objetivo fue reducir el trabajo manual necesario para expandir el dataset. Estas máscaras no deben considerarse anotaciones finales, sino insumos editables para una posterior revisión manual o experta.

Una ventaja importante de las imágenes generadas es que facilitan la edición posterior: al separar las estructuras por color o por máscara binaria, un evaluador puede revisar las regiones de forma individual, corregir bordes, eliminar regiones incorrectas o validar la columna completa antes de convertirla en una etiqueta definitiva.

Por lo tanto, el flujo propuesto permite documentar y acelerar la expansión del dataset, pero las decisiones anatómicas dudosas deben ser revisadas por un experto clínico antes de utilizarse como ground truth definitivo.